

Introduction à Node.js

Node.js est un environnement d'exécution JavaScript côté serveur qui permet aux développeurs de créer des applications web évolutives et performantes. Contrairement à JavaScript traditionnel qui s'exécute dans le navigateur, Node.js s'exécute sur le serveur, ce qui permet de gérer des opérations d'entrée/sortie (I/O) de manière non bloquante et asynchrone.

La session prochaine couvrira l'utilisation de Node.js pour créer des applications web, des API RESTful, et bien plus encore. Pour l'instant, nous nous en servons pour compiler notre code en utilisant des outils de développement front-end comme Vite et installer des paquets via npm.

Caractéristiques principales de Node.js

- **Asynchrone et non bloquant** : Node.js utilise un modèle d'E/S non bloquant, ce qui signifie qu'il peut gérer plusieurs connexions simultanément sans attendre que les opérations d'E/S soient terminées.
- **Basé sur le moteur V8 de Google** : Node.js utilise le moteur JavaScript V8 de Google, qui compile le code JavaScript en code machine natif, offrant ainsi des performances élevées.
- **Gestion des modules** : Node.js dispose d'un système de modules intégré qui permet aux développeurs de réutiliser du code et de gérer les dépendances via le gestionnaire de paquets npm (Node Package Manager).
- **Écosystème riche** : Grâce à npm, Node.js bénéficie d'un vaste écosystème de bibliothèques et de frameworks qui facilitent le développement d'applications web, d'API, de microservices, etc.

Node Package Manager (npm)

npm est le gestionnaire de paquets par défaut pour Node.js. Il permet aux développeurs d'installer, de partager et de gérer des bibliothèques et des outils tiers pour leurs projets Node.js.

Installer Node.js

Vous pouvez télécharger et installer Node.js depuis le site officiel : <https://nodejs.org/>. Choisissez la version LTS (Long Term Support) pour une stabilité optimale. Par défaut, npm est inclus avec l'installation de Node.js vous n'avez donc pas besoin de l'installer séparément.

Si votre système utilise un gestionnaire de paquets comme Homebrew (macOS) ou apt-get (Linux), vous pouvez également installer Node.js via ces outils.

Si vous aviez un terminal ouvert avant l'installation de Node.js, fermez-le et rouvrez-le pour que les modifications soient prises en compte.

Initialiser un projet Node.js

Pour créer un nouveau projet Node.js, allez à la racine de votre projet dans le terminal et exécutez la commande suivante :

```
npm init
```

Cela lancera un assistant qui vous posera plusieurs questions pour configurer votre projet (nom, version, description, point d'entrée, etc.). Vous pouvez appuyer sur Entrée pour accepter les valeurs par défaut ou fournir vos propres réponses. À la fin, un fichier `package.json` sera créé dans le répertoire de votre projet.

Package.json

Le fichier `package.json` est un fichier de configuration essentiel dans les projets Node.js. Il contient des informations sur le projet, telles que son nom, sa version, ses dépendances, ses scripts, et d'autres métadonnées. Il est créé automatiquement lorsque vous initialisez un projet avec `npm init`. Les dépendances sont les bibliothèques ou les modules dont votre projet a besoin pour fonctionner correctement. Elles sont listées dans la section `dependencies` du fichier `package.json`. Les dépendances de développement, quant à elles, sont des outils nécessaires uniquement pendant le développement (comme les outils de test ou de compilation) et sont listées dans la section `devDependencies`.

```
{
  "name": "mon-projet",
  "version": "1.0.0",
  "description": "Une description de mon projet",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Votre Nom",
  "license": "ISC",
  "dependencies": {
    "express": "^4.17.1"
  },
  "devDependencies": {
    "jest": "^26.6.3"
  }
}
```

Commandes npm courantes

- `npm init` : Initialise un nouveau projet Node.js en créant un fichier `package.json`.
- `npm install` : Installe toutes les dépendances du projet qui sont listées dans le fichier `package.json`. Utile lorsque vous clonez un projet existant.
- `npm install <package>` : Installe un paquet et l'ajoute aux dépendances du projet.
- `npm install -D <package>` : Installe un paquet en tant que dépendance de développement.
- `npm install -g <package>` : Installe un paquet globalement sur le système et non uniquement dans le projet.
- `npm uninstall <package>` : Désinstalle un paquet du projet.
- `npm update` : Met à jour les paquets installés vers leurs dernières versions.

Attention Lorsque vous installez des paquets et que vous déployez votre projet avec GitHub, n'oubliez pas d'ajouter le dossier `node_modules` à votre fichier `.gitignore` pour éviter de versionner les dépendances. Le fichier `package.json` contient toutes les informations sur les dépendances de votre projet, ce qui permet à d'autres développeurs de recréer l'environnement en utilisant la commande `npm install`.

Installer des paquets de développement

On utilise souvent des paquets de développement pour des tâches spécifiques comme la compilation, le test, ou le linting. Pour installer un paquet en tant que dépendance de développement, utilisez l'option `-D` ou `--save-dev` avec la commande `npm install`. Lorsque les paquets sont installés en tant que dépendances de développement, ils sont ajoutés à la section `devDependencies` du fichier `package.json`. Lors de la compilation ou du déploiement de l'application, ces paquets ne sont pas inclus, ce qui permet de réduire la taille finale de l'application.

```
npm install -D <package>
```

```
npm install --save-dev <package>
```

Exécuter des scripts définis dans package.json

Vous pouvez définir des scripts personnalisés dans le fichier `package.json` sous la section `scripts`. Par exemple, vous pouvez ajouter un script pour démarrer votre application ou pour exécuter des tests. Voici un exemple de section `scripts` dans un fichier `package.json`. Dans cet exemple, nous avons défini des scripts pour démarrer le serveur de développement (`dev`), compiler le projet pour la production (`build`), prévisualiser la version de production (`serve`), et exécuter les tests (`test`). Nous verrons `Vite` et `Jest` plus loin.

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build",  
  "serve": "vite preview",  
  "test": "jest"  
},
```